ASSIGNMENTS

SET A: Human Face Detection & Recognition System (FRS)

Goal
Design and implement an end-to-end Face Recognition Service (FRS) that detects faces in CCTV frames, extracts embeddings, and recognizes identities from a small gallery. The service must be production-ready as a microservice (FastAPI + Docker) and tuned for CPU inference.

Dataset

- Recommended: use public face datasets such as **WIDER FACE** (for detection) + **VGGFace2** or **MS-Celeb-1M (cleaned subsets)** for training/fine-tuning embeddings
- If restricted, candidate may create a small in-house gallery dataset (20–100 identities, 5–10 images per identity) captured under varied lighting/angles.

Tasks

1. **Data prep**
    - Download / collect dataset subset; ensure train/val splits.
    - Perform face cropping, alignment (5-point), normalization.
2. **Face detection**
    - Implement RetinaFace / YOLO face detector (use pretrained weights
    - Provide detection metrics (precision/recall on a small validation set).
3. **Feature extractor**
    - Use pretrained embedding model (AdaFace/ArcFace/FaceNet). Fine-tune on gallery if desired.
    - Save embeddings in a database (SQLite/Postgres) with metadata (id, name, image_path, timestamp).
4. **Matching pipeline**
    - Implement cosine similarity / L2 nearest neighbor search.
    - Add configurable threshold and top-K return.
    - Optional: implement Faiss index for faster retrieval.
5. **Microservice**
    - FastAPI endpoint(s): `/detect`, `/recognize`, `/add_identity`, `/list_identities`.
    - Output: bounding boxes, identity (if matched), confidence.

    - Include Dockerfile and `requirements.txt`.

6. **Optimization**
    - Convert model to ONNX / TorchScript, benchmark CPU latency (ms) and throughput (FPS).
    - Apply post-processing to reduce false positives (NMS thresholds, face quality filter).

7. **Evaluation & robustness**
    - Report accuracy (precision/recall), identification rate (top-1, top-5), latency on CPU.
    - Discuss failure modes (occlusions, low-light) and mitigations.

Helpful resources

- RetinaFace, ArcFace, AdaFace repos (official implementations)
- Faiss (for index/search)
- OpenCV face alignment tutorials

Deliverables

- Code repo or zipped folder (notebook + code + Dockerfile).
- Docker image or instructions to build.
- Short report (PDF/MD) with methodology, accuracy numbers, CPU benchmarks, and limitations.
- API documentation (Swagger or Postman collection).
- Optional: small demo video/gif showing recognition on CCTV sample frames.